

PYTHON基础教程

简介

欢迎阅读本教程，本教程旨在指导学生掌握Python基础。在数据驱动的编程世界中，文本数据处理是必不可少的技能，尤其是在日常工作中频繁遇到文本格式化、数据清洗和信息解析的任务。Python由于其优雅的语法、强大的内作功能和丰富的生态，被广泛应用。

学习PYTHON的价值

Python的设计哲学强调代码的可读性和简洁性，其允许开发者用较少的代码行实现功能强大的解决方案。随着Python在科学计算、机器学习、网络服务和自动化等领域的应用越来越广泛，掌握Python将为个人职业发展提供宝贵的优势。

教程目标和结构概览

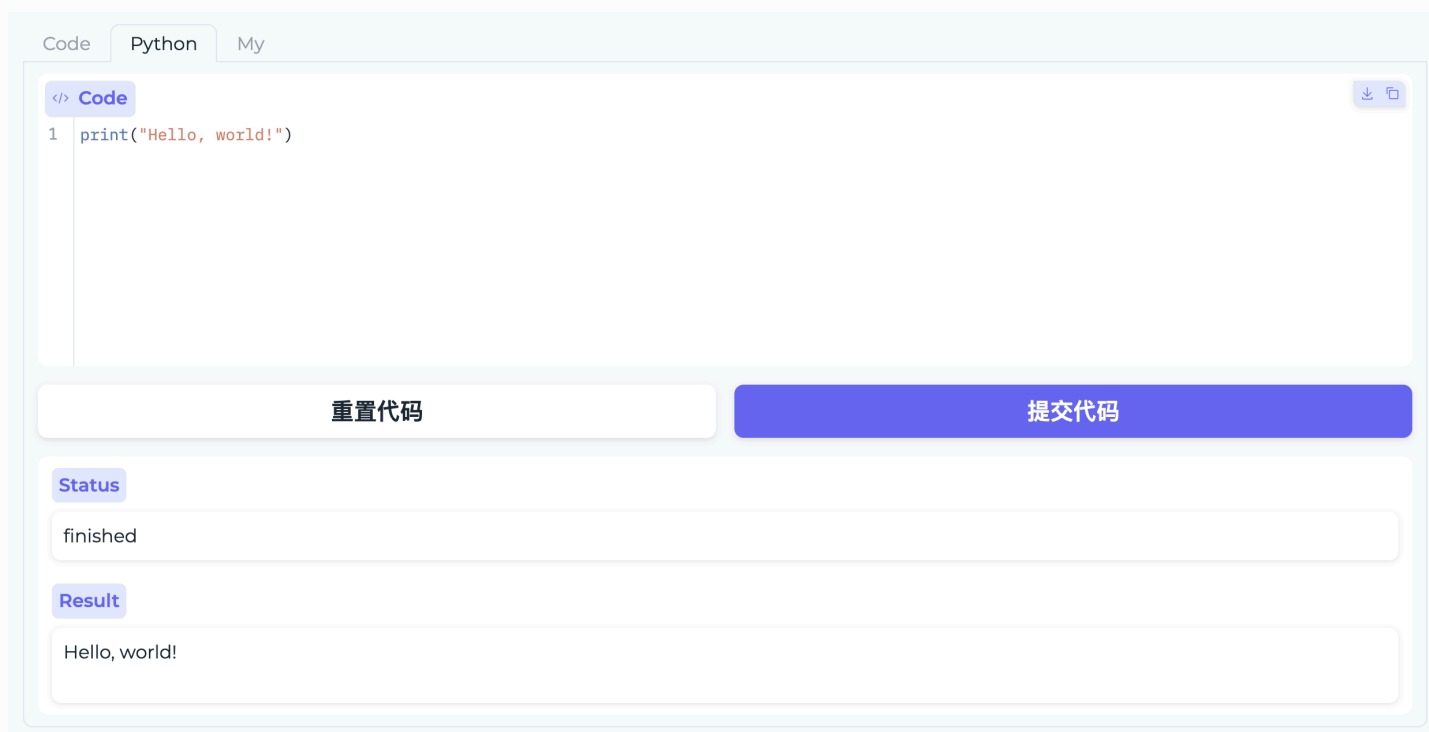
在本教程中，我们介绍Python的核心编程概念。我们先介绍Python中的代码块和缩进。然后，介绍Python中的变量。之后，我们介绍Python中几个基础的数据类型。最后我们介绍Python中的基本语句，和函数的基础概念。

第一个PYTHON程序

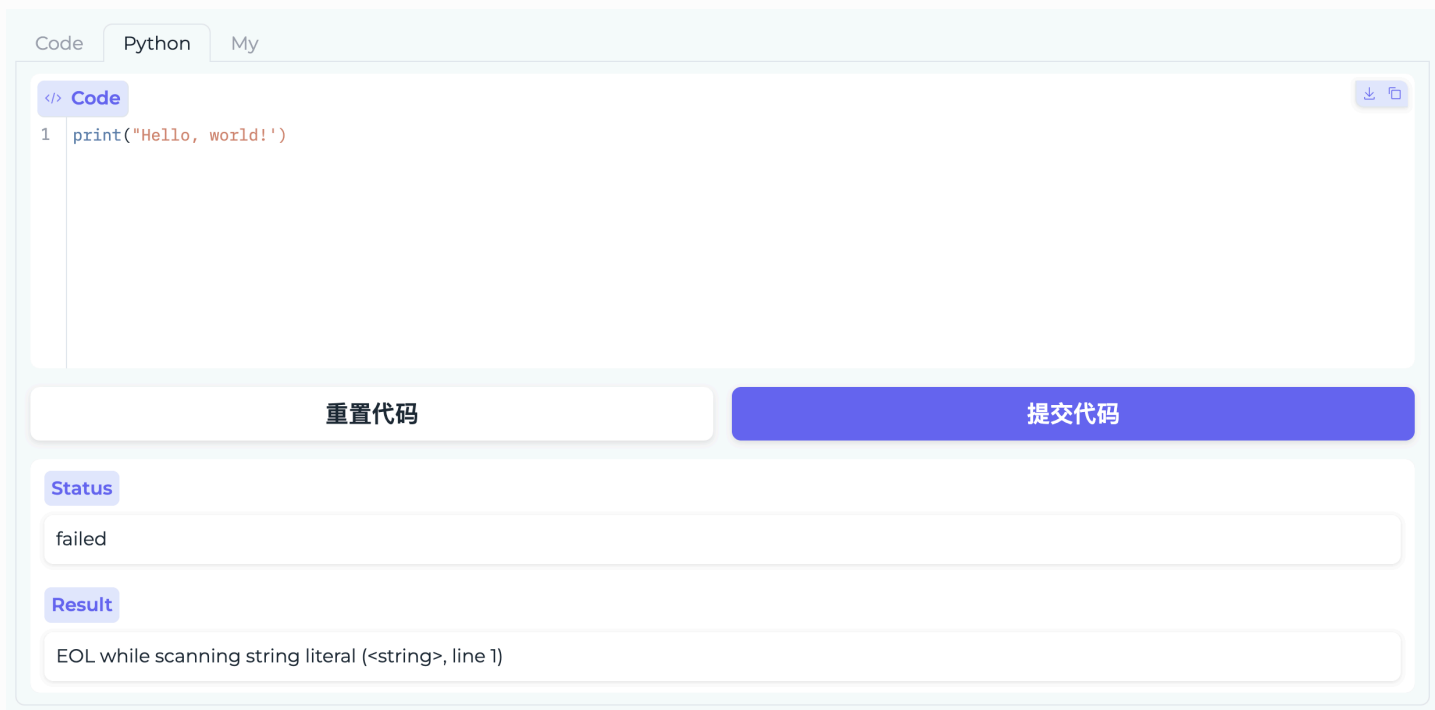
我们第一个介绍的Python程序是打印 `Hello, world!` 字符串。如果要让Python打印出指定的文字，可以用 `print()` 函数，然后把希望打印的文字用单引号或者双引号括起来，但不能混用单引号和双引号。

```
print('Hello, world!')
```

在本次实验中，我们为大家提供了一个Python可执行环境。如下图所示，大家可以在程序框中输入想要执行的代码（如上介绍的打印 `Hello, world!` 小程序）。点击运行，界面会展示程序中要求打印的输出。



或者当程序出现错误时，结果界面会展示程序的报错信息。



代码块和缩进

Python的语法比较简单，特别强调**代码块**和**缩进**。下面是一个Python代码的例子

```
name = "world"
if name == "world":
    greet_str = "Hello world!" # 属于if语句的代码块
    print("greet_str") # 属于if语句的代码块
```

以 `#` 开头的语句是注释，注释是给人看的，可以是任意内容，解释器会忽略掉注释。其他每一行都是一个语句，当语句以冒号 `:` 结尾时，缩进的语句视为代码块。

最后，请务必注意，Python程序是大小写敏感的，如果写错了大小写，程序会报错。

记得：在Python中，缩进是必须严格遵守的规则，通常使用四个空格作为标准缩进。

变量

在Python中，**变量** 就像是一个盒子，你可以在里面存储各种信息。

动态类型

Python 是一种动态类型语言，变量在声明时不需要显式指定数据类型。Python 解释器在运行时自动推断变量的类型。

例如，在赋值时无需指定变量的类型

```
x = 10          # x 是整数
x = "Hello"     # 现在 x 是字符串
x = 3.14        # 现在 x 是浮点数
```

定义函数参数时也无需指定变量的类型（之后的章节会介绍Python中的函数）

```
# 一个将a和b相加的函数
def add(a, b):
    return a + b
```

变量的命名规则

变量的命名和使用也有一些规则和最佳实践：

1. **变量名的开始**：变量名必须以字母或下划线 `_` 开头，不能以数字开头。
2. **字符的使用**：变量名只能包含字母、数字和下划线（A-Z, a-z, 0-9, 和 `_`）。

3. **区分大小写**：Python中的变量名是区分大小写的。
4. **避免关键字**：不能使用Python的保留关键字作为变量名，如 `if` , `while` , `class` 等。
5. **可读性**：选择有意义的名称，使代码更易于理解。

一些有效的变量名的例子：

```
username = "admin"  
counter = 10  
max_height = 200  
_profile = "private"  
VERSION = "1.0.0"
```

一些无效的变量名的例子：

```
2users = "Bob and Alice"    # 不能以数字开头  
user-name = "admin"         # 不能包含除下划线以外的特殊字符  
global = "value"            # 不能使用关键字  
class = "12A"                # 不能使用关键字  
a/b = 5                      # 不能包含特殊字符
```

变量的定义

在python中，直接通过赋值来定义一个变量。使用未赋值的变量，会引发错误。

注：这和C语言中可以声明一个变量而不立即初始化是有显著不同的。

下面是一个例子：

```
# x未定义
print(x) # 报错: NameError

# 定义x为一个空字典
x = {}
print(x) # 输出{}
```

数据类型

计算机可以处理各种不同类型的数据。不同的数据需要定义不同的数据类型。Python可以直接处理的数据类型有以下几种:

数值

浮点数和整数

Python中常用的数值类型有整数和浮点数。Python可以处理任意大小的整数，当然包括负整数

```
x = 10
y = -5
z = 0
```

浮点数用于表示带有小数点的数值。Python中的浮点数采用IEEE 754标准表示，因此可以表示非常大或非常小的数。例如：

```
a = 3.14
b = -0.5
c = 1.23e-5 # 科学计数法表示
```

简单数值运算

Python支持各种数值计算，包括加法、减法、乘法和除法等。

加法（`+`）和减法（`-`）相当直用于计算两个数的和或差：

```
summation = 7 + 3 # 结果为 10
difference = 10 - 2 # 结果为 8
```

乘法（`*`）和除法（`/`）用于计算两个数的乘积或商：

```
product = 4 * 5 # 结果为 20
quotient = 20 / 4 # 结果为 5.0
```

Python还支持整除（`//`）和求余（`%`）运算。整除会返回两个数相除的整数部分，而求余运算会返回除法后的余数：

```
integer_division = 8 // 3 # 结果为 2
remainder = 8 % 3 # 结果为 2
```

最后，指数运算（`**`）用于计算一个数的另一个数次幂：

```
exponentiation = 2 ** 3 # 结果为 8
```

空值

空值是Python里一个特殊的值，用 `None` 表示。`None` 不能理解为 `0`，因为 `0` 是有意义的，而 `None` 是一个特殊的空值。

字符串

Python中的字符串是由字符组成的序列。它们可以被用于存储和表示文本信息。字符串可以用单引号(' ')或双引号(" ")来创建。

```
string1 = 'Hello'
string2 = "World"
```

使用 `len()` 函数可以找出字符串中的字符数量。

```
length = len('Hello') # 结果为5
```

连接和重复

在Python中，你可以使用 `+` 来连接（拼接）两个字符串，或使用 `*` 来重复一个字符串多次。

```
string1, string2 = 'Hello', "World"
combined_string = string1 + string2 # 结果为'HelloWorld'
```

```
string1 = 'Hello'
repeated_string = string1 * 3 # 结果为'HelloHelloHello'
```

索引和切片

字符串索引允许你访问特定的字符，而切片则允许你获取字符串的一个子集。

- **索引**：Python中的字符串索引从0开始。负数索引从字符串的末尾开始计数，-1指的是字符串的最后一个字符。格式为 `string[index]`。下标超出字符串长度会报错。


```
string1 = 'Hello'
first_char = string1[0] # 结果为'H'
last_char = string1[-1] # 结果为'o'

# IndexError: string index out of range
out_char1 = string1[10]
out_char2 = string2[-10]
```

- 切片：通过指定开始和结束索引来工作，格式为 `string[start:end:step]`。
 - `start` 表示切片的开始位置，会被包含在切片的结果中。如果省略 `start`，则默认从字符串开始切片；负数 `start` 表示从末尾开始计数；
 - `end` 是切片的结束位置，不会被包含在切片的结果中。如果省略 `end`，则一直切片到字符串的末尾，这个时候字符串末尾会被包含。负数 `end` 表示从末尾开始计数；
 - `step` 是步长，表示切片中每个元素之间的间隔，默认为1。负数 `step` 会导致字符串被反向遍历。

注：

- 切片操作不会修改原字符串，而是生成一个新的字符串。
- 如果 `start` 或 `end` 指定的位置超出了字符串的长度，Python不会报错，而是尽可能地返回一个结果。
- `step` 不能为0，否则会引发 `ValueError`。

```
string1 = "Hello, world!"

# 基础切片
print(string1[1:4]) # 'ello' 从第2到第4个字符（下标为4的字符不被包含）

# 省略开始和结束：如果省略开始，则默认从头开始；如果省略结束，则切片到字符串末尾
print(s[7:]) # 'world!' 从第8个字符到结束
print(s[:5]) # 'Hello' 从开始到第5个字符（下标为5的字符不被包含）

# 负数索引：使用负数索引可以从字符串的末尾开始计数
```

```
print(s[-6:-1]) # 'world' 到数第6个字符到倒数第二个字符

# 步长：步长决定了切片获取字符的间隔
print(s[1:10:2]) # 'el,wr' 从第2个字符（e）到第10个字符（r），每2个字符取1个

# 完整切片
print(s[::]) # 'Hello, world!'

# 反向切片
print(s[::-1]) # '!dlrow ,olleH'
```

列表

Python中的列表用于存储一系列的元素。

列表用方括号 `[]` 定义，可以包含不同类型的元素。可以使用 `len()` 函数获取列表的长度。

```
my_list = [1, 2, 3, 'Python', 'AI']
list_length = len(my_list) # 结果为 5
```

索引和切片

与字符串相似，可以使用索引来访问列表中的元素，或者使用切片来获取列表的一部分。

```
my_list = [1, 2, 3, 'Python', 'AI']
first_element = my_list[0] # 结果为 1
slice_of_list = my_list[1:4] # 结果为 [2, 3, 'Python']
```

添加和删除元素

- `append()` 用于在列表末尾添加元素。
- `insert()` 可以在指定位置插入元素。
- `pop()` 用于移除列表中的一个元素（默认是最后一个），并返回该元素的值。

```
my_list = [1, 2, 3, 'Python', 'AI']
my_list.append('new') # 添加 'new' 到列表末尾
my_list.insert(2, 'inserted') # 在索引2的位置插入 'inserted'
popped_element = my_list.pop() # 移除并返回最后一个元素
```

字典

字典是一种通过键来存取数据的数据结构，每个键映射到一个值。键必须是唯一的，而值则可以是任何数据类型。字典使用大括号 `{}` 定义，通过键来存取数据。可以使用 `len` 函数获取字典的元素个数。

```
# 创建字典
my_dict = {'name': 'Alice', 'age': 25, 'city': 'New York'}
print(len(my_dict)) # 长度为3

# 访问字典中的值
print(my_dict['name']) # 输出 'Alice'
```

如果试图访问字典中不存在的键，则会引发一个 `KeyError`。

```
my_dict['address'] # KeyError: 'address' not found in dictionary
```

在尝试访问字典中的值之前，可以 `in` 运算符检查键是否存在于字典中，以避免 `KeyError`。

```
if 'name' in my_dict:
    print(my_dict['name']) # 安全地访问
```

添加和修改元素

通过制定已有或新键对应的值，来添加或修改元素

```
my_dict['age'] = 26 # 修改已有的键
my_dict['email'] = 'alice@example.com' # 添加新的键值对
```

删除元素

使用 `del` 语句或 `pop` 方法可以删除字典中的元素。

```
del my_dict['city'] # 删除一个键值对
# 或者
email = my_dict.pop('email') # 移除键 'email' 并获取其值
```

布尔值

在 Python 中，布尔值是表示真或假的数据类型。取值有两个值：`True` 和 `False`。

比较运算

比较运算符用于比较两个值，并返回一个布尔值（`True` 或 `False`）。Python支持多种比较运算符：

- 等于（`==`）
- 不等于（`!=`）
- 小于（`<`）

- 大于 (`>`)
- 小于等于 (`<=`)
- 大于等于 (`>=`)

这些比较运算符不仅适用于数值，还可以用于字符串、列表和其他序列类型的比较。我们在之后的章节会介绍这些数据类型。不同类型的比较逻辑不同：

- **数值比较**：当使用比较运算符比较数值时，比较的是数值的大小。例如，`3 < 4` 会评估为 `True`，因为3小于4。
- **字符串比较**：字符串的比较是基于字典顺序的。首先比较两个字符串的第一个字符，如果不同，则比较结果由这两个字符决定。如果相同，则继续比较下一个字符，依此类推。例如，`"apple" < "banana"` 会评估为 `True`，因为字母 `"a"` 在字母 `"b"` 之前。
- **列表比较**：列表的比较类似于字符串比较。首先比较两个列表的第一个元素，然后是第二个元素，依此类推，直到找到不同的元素。如果一个列表的元素是另一个列表的前缀，则较短的列表被认为较小。例如，`[1, 2, 3] < [1, 2, 4]` 会评估为 `True`，因为在第三个元素上3小于4。

以下是一些例子：

```
print(5 == 5)    # True
print(5 != 2)    # True
print(3 < 4)     # True
print(6 > 7)     # False
print("apple" == "apple") # True
print("apple" < "banana") # True
print("app" > "apple")    # False
print([1, 2, 3] < [1, 2, 4]) # True
print([1, 2, 3] == [1, 2, 3]) # True
print([1, 2] == [1, 2])      # True
print([1, 2] < [1, 2, 3])    # True
```

逻辑运算

Python 中的逻辑运算符用于组合布尔值（`True` 或 `False`）并返回一个布尔值。主要有三种逻辑运算符：

- `and`：如果两个布尔值都为 `True`，则返回 `True`；否则返回 `False`。它用于确保两个（或多个）条件同时满足。
- `or`：如果至少有一个布尔值为 `True`，则返回 `True`；如果两个都是 `False`，则返回 `False`。它用于检查至少一个条件是否满足。
- `not`：用于反转布尔值。如果布尔值是 `True`，则返回 `False`；如果是 `False`，则返回 `True`。

以下是一些逻辑运算的例子：

```
print(True and True)    # True
print(True and False)   # False
print(False and False)  # False
print(True or False)    # True
print(False or False)   # False
print(not True)         # False
print(not False)        # True

# 结合比较运算符和逻辑运算符
x = 5
y = 8
print(x > 3 and y < 10) # True
print(x > 3 and y > 10) # False
print(x < 3 or y < 10)  # True
print(not(x > 3))       # False
```

逻辑运算符的优先级低于比较运算符，这意味着在表达式中，首先会进行比较运算，然后再进行逻辑运算。因此，在上述例子中，先计算了 `x > 3` 和 `y < 10`，然后计算了这两个布尔值的逻辑 `and`。

布尔值在条件语句（如 `if` 语句）中非常有用。例如下面这个例子中当 `x > 3` 为真时，会输出大于 3。我们将在下一个章节介绍条件语句。

```
if x > 3:
    print("大于 3")
```

身份和成员运算符

除了基本的比较运算符，Python 还提供了其他类型的运算符来检查变量的身份和成员关系，主要包括 `is` 和 `in` 运算符。

`is` 运算符

运算符 `is` 用于比较两个对象的身份，即它们是否为同一个对象，这涉及比较对象的内存地址。如果两个变量引用的是同一个对象，则 `is` 运算符返回 `True`；否则返回 `False`。

在使用 `is` 运算符时，需要注意它与 `==` 运算符的区别。`==` 用于比较两个对象的值是否相等，而 `is` 检查的是两个对象的身份是否相同。

```
x = [1, 2, 3]
y = x
z = [1, 2, 3]
print(x is y)    # True, 因为 x 和 y 指向同一个列表
print(x is z)    # False, x 和 z 指向不同的对象
```

`in` 运算符

运算符 `in` 用于检查一个元素是否存在于一个序列中（如列表、元组、字符串等）。如果元素在指定的序列中，`in` 运算符返回 `True`；否则返回 `False`。

以下是使用 `in` 运算符的一些例子：

```
# 列表
my_list = [1, 2, 3, 4, 5]
```

```
print(3 in my_list)  # True, 因为 3 在列表 my_list 中
print(6 in my_list)  # False, 因为 6 不在列表 my_list 中

# 字符串
my_string = "hello world"
print('world' in my_string)  # True, 因为子字符串 'world' 在字符串
my_string 中
print('python' in my_string)  # False, 因为子字符串 'python' 不在字符串
my_string 中

# 字典
my_dict = {'name': 'Alice', 'age': 30}
print('name' in my_dict)  # True, 因为键 'name' 在字典 my_dict 中
print('Alice' in my_dict)  # False, 因为 'Alice' 是一个值而不是键
```

基本语句

赋值

在上述教程中其实已经涉及很多赋值语句的。在 Python 中，赋值语句用于创建新变量或更新现有变量的值。赋值语句的基本形式是将一个值（或表达式的结果）分配给一个变量名。

```
x = 10          # 将整数 10 赋值给变量 x
y = x + 5       # 将 x 和 5 的和赋值给变量 y
text = "Hello, Python!"  # 将字符串赋值给变量 text
```

由于Python是动态类型语言，同一个变量可以前后赋值不同类型的数据。


```
x = 5
x = "Hello, wolrd" # x先是整数, 后为字符串
```

条件判断

条件判断语句用于基于一个或多个条件执行不同的代码块。最常用的是 `if` 语句。

`if` 语句用于测试一个条件，如果条件为真，则执行冒号后面缩进的代码块。

```
# 年龄20大于18, 会打印adult
age = 20
if age >= 18:
    print('your age is', age)
    print('adult')
```

也可以给 `if` 添加一个 `else` 语句

```
# 年龄8小于18, 会打印teenager
age = 8
if age >= 18:
    print('your age is', age)
    print('adult')
else:
    print('your age is', age)
    print('teenager')
```

注意不要少写冒号。当条件区间更多时，可以使用 `elif` 做更细致的判断：

```
# 年龄3小于6, 会打印kid
age = 3
if age >= 18:
    print('adult')
elif age <= 6:
    print('kid')
else:
    print('teenager')
```

所以 `if` 语句的完整形式是（`elif` 和 `else` 可以省略）

```
if <条件判断1>:
    <执行1>
elif <条件判断2>:
    <执行2>
elif <条件判断3>:
    <执行3>
...
else:
    <执行4>
```

循环语句

循环用于重复执行代码块，直到达成特定条件。Python 提供了两种主要的循环语句：`for` 循环和 `while` 循环。

`for` 循环

`for` 循环用于遍历任何序列（如列表、字符串、字典），对序列中的每个元素执行代码块。格式为：

```
for <元素> in <序列>:
    <代码块>
```

循环遍历不同序列

- 用 `for` 遍历列表时，会对每一个列表的元素执行循环冒号后代码块的操作
- 用 `for` 遍历字符串时，会对每一个字符串中的字符依次执行循环冒号后代码块的操作
- 用 `for` 遍历字典时，会对字典的每一个键值依次执行循环冒号后代码块的操作

```
# 遍历列表
my_list = [1, 2, 3, 4, 5]
# 依次打印出1、2、3、4、5
for elem in my_list:
    print(elem)

# 遍历字符串
my_string = "hello world"
# 依次打印出'h'、'e'、'l'、'l'、'o'、' '、'w'、'o'、'r'、'l'、'd'
for elem in my_string:
    print(elem)

# 遍历字典
my_dict = {'name': 'Alice', 'age': 30}
# 依次打印出'name'和'age'
for elem in my_dict:
    print(elem)
```

例子：计算序列和

下面是计算1-3的和的一个例子：

```
# 计算1-3的和
result = 0
for i in [1, 2, 3]:
    result = result + i
print(result)
```

但是当我们想要计算1-100整数和呢，手写一个1-100的列表可能有些困难。

Python提供一个 `range()` 函数，可以生成一个整数序列。`range(start, end, step)` 函数可以生成从 `start` 到 `end-1` 步长为 `step` 的序列。其中 `start` 省略则默认为0。`step` 省略则默认为1，负数 `step` 表示递减序列。`end` 不可省略。

再通过 `list()` 函数可以将 `range` 的输出转换为列表。比如 `range(5)` 可以生成从0到4的整数列表

```
list(range(5)) # [0, 1, 2, 3, 4]
```

`range(101)` 就可以生成0-100的整数序列，计算如下：

```
result = 0
for x in range(101):
    result = result + x
print(result)
```

`while` 循环

`while` 循环在满足特定条件时不断执行一个代码块。格式为：

```
while <条件判断>:
    <代码块>
```

计算1-100整数和也可以用 `while` 循环实现。示例：

```
result = 0
x = 0
while x < 101:
    result += x
    x += 1
```

循环控制语句

`break`：用于提前退出循环。下面的例子中，当 `i` 等于 5 时，`break` 语句执行，循环终止。

```
# for 循环
for i in range(10):
    if i == 5:
        break
    print(i)

# while 循环
i = 0
while i < 10:
    if i == 5:
        break
    print(i)
    i += 1
```

`continue`：跳过当前循环的剩余部分，直接开始下一次循环迭代。下面的例子中，当 `i` 是偶数时，`continue` 语句会跳过循环的剩余部分，因此只有0-9的奇数被打印。

```
# for 循环
for i in range(10):
    if i % 2 == 0:
        continue
    print(i)

# while 循环
i = 0
while i < 10:
    if i % 2 == 0:
        continue
    print(i)
    i += 1
```

函数基础

在完成作业时，我们会要求使用函数的形式完成对应内容。Python中的函数通常使用 `def` 关键字来定义。函数的基本结构如下：

```
def 函数名(参数列表):  
    函数体  
    return 返回值
```

- **函数名**：这是函数的名字，用于之后调用这个函数。
- **参数列表**：函数可以接受零个或多个参数，这些参数在函数被调用时传递给函数。
- **函数体**：这是执行具体操作的代码块。
- **返回值**：函数可以返回一个值。如果没有返回值，可以省略 `return` 语句或写 `return None`。

示例：

这个函数 `greet` 接受一个参数 `name`，并返回一个问候语字符串。

```
def greet(name):  
    return "Hello, " + name + "!"  
  
print(greet("Alice")) # 输出: Hello, Alice!  
print(greet("Tom"))  # 输出: Hello, Tom!
```

这个函数 `sum_minus` 接受两个参数 `x` 和 `y`，返回 `x` 和 `y` 的和以及 `x` 和 `y` 的差

```
def sum_minux(x, y):  
    return x+y, x-y  
  
print(sum_minux(3, 1)) # 输出: 4, 2  
print(sum_minux(8, 3)) # 输出: 11, 5
```